Design Rationale

a) Identified Weakness

In the original design, the ProductService class directly depended on a concrete Impact class. This created a clear violation of both the Open/Closed Principle (OCP) and the Dependency Inversion Principle (DIP). Since ProductService was tightly coupled to a specific implementation, any new way of calculating impact required modifying the existing class.

This design leads to a fragile system. For example, if we wanted to introduce a new impact calculation method (such as carbon-weighted impact or region-based impact), we would have to go back and change the logic inside ProductService. This increases the risk of breaking existing functionality and makes the code harder to maintain.

A concrete example of a painful change request would be: “Add a new impact calculation based on recycling efficiency over time.” In the original design, implementing this would require editing ProductService, testing all dependent features again, and possibly introducing bugs.

b) Principle Applied

To solve this, the design applies the Open/Closed Principle (OCP) and Dependency Inversion Principle (DIP).

OCP means that the system should be open for extension but closed for modification. In this context, it means we should be able to add new impact calculation methods without changing existing code.

DIP means that high-level modules (like ProductService) should not depend on low-level modules (like a concrete Impact class), but instead depend on abstractions.

These principles matter because the system is expected to evolve. New impact strategies will likely be added over time, and the design must support that without constant rewriting of core logic.

c) Refactoring Move

The key structural change was introducing an interface called ImpactCalculationStrategy. Instead of depending on a concrete class, ProductService now depends on this interface.

Two implementations were created:

- SimpleSumStrategy

- WeightedByLifespanStrategy

Additionally, constructor injection was introduced. This means the strategy is passed into ProductService when it is created, rather than being hard-coded inside it.

The Main class now acts as the composition root, where the actual strategy is selected and wired into the system.

This resolves the violation because ProductService is no longer tied to any specific implementation.

d) Design Impact

After refactoring, adding new strategies is much easier. You simply create a new class implementing the interface—no need to modify existing code.

Testing is also improved. Each strategy can be tested independently, and ProductService can be tested using mock strategies.

Finally, the system now supports new capabilities without modification. For example, adding advanced impact models or experimenting with different algorithms becomes straightforward.

Overall, the design is now more flexible, maintainable, and aligned with SOLID principles.